

# Lecture 1

## Introduction to Operating System

↳ Software ni anjgali software

software    ဘဲ တိတ်ကွပ် hardware သက် OS

↓                                          ↓

Run လိုသ် `printf("Hello World!");`    ပါ့ OS က monitor  
ပါ့ပေန် Hello World

## 🧠 What's the problem?

When programs (like apps you use) need to do something dangerous or powerful, like reading a file from your hard drive or sending data over the internet, they can't do it directly. That's because these actions could harm the computer if not done carefully.

## 🔒 Two Modes of Operation

Your computer has two "modes":

**User Mode** – for regular programs (like games, browsers, etc.)  
→ Limited access: can't directly talk to hardware (like disk or network).

**Kernel Mode** – for the Operating System (OS)  
→ Full access to the computer's hardware.

## 🔄 What is a System Call?

A system call is a special way for a program in user mode to ask the OS (in kernel mode) to do something on its behalf.

Think of it like this:

"Hey OS! I need to read this file. I can't do it myself, but you can!"

## ⚙️ How It Works (Step by Step):

- Your program wants to do something (like read a file).
- It makes a system call (via a special instruction, called a trap).

The hardware:

- Switches to kernel mode.
- Jumps to a special OS function called a trap handler.
- The OS does the work (e.g., reads the file).

When done, the OS uses another special instruction to:

- Return control to your program.
- Switch back to user mode.

## 💡 Key Difference:

A regular procedure call just jumps to another function in your program (nothing fancy).

A system call is special: it switches control and power to the OS so it can safely do things that user programs are not allowed to do directly.

"System Call!  
Generate Luminous Element. Adhere!"  
- Alice -



# From previous semester ...

- Every second:
  - Millions instructions are fetched by a processor (from a memory),
  - then decoded (processor figure out what instruction it is),
  - then executed (and maybe write)
- In this class, we will learn that while a program is run, a lot of other things are going on...and
- There is a body of software that is responsible for
  - making programs run
  - Allowing programs to share memory
  - Enabling programs to interact with devices
  - Etc.

Operating System is in charge of making sure that the system operates correctly and efficiently in an easy-to-use manner

# Things covered in this course

## Lectures

- Introduction to OS (1 wk)
- Part I – **Virtualization** (5 wks)
  - Processor จำลอง CPU ให้เหมือนจริง ๆ ที่จริงแล้ว CPU หนึ่ง
  - Memory software จะใช้หน่วยร่วมกันได้
  - Memory เหมือนในเหมือนจริง mem ไม่ซ้ำใคร
- Part II – **Concurrency** (4 wks) ทำงานพร้อมกัน
  - Thread API
  - Locks and Semaphores
  - Deadlock
- Part III – **Persistence** (2 wks)
  - I/O System
  - File System
  - Storage and Mass-Storage Management

## Mini Projects

- Project 1 – Compile and build an OS
- Project 2 – Study and present in details, a few key components of an OS
- Project 3 – Modify some modules in the kernel

# Virtualization

- OS takes a physical resources (processor, memory, disk, etc) and transform it into a more general, powerful *virtual* form of itself.
  - Ex OS can turn a single CPU into a seemingly infinite number of CPUs – **virtualizing the CPU**
- Sometime, we refer to OS as a ***virtual machine***
- OS allows CPUs, memory, I/O to be shared, thus it can also be called a ***resource manager***.
- OS provides **APIs** (system calls) to applications in order to access resources.

Application Programming Interface

Software applications with API

vs on hardware software with keyboard, monitor

# Processor

- Q: If two programs want to run at the same time, then what ?
- Q: Who (which program) gets what (which resources), and when ?
  - <sup>(Algorithm)</sup> **Policies**, many and many of them *ໂປລິທິກດີວ໌ ກໍ 2 ປຣອແກມ ທີ່ຕາມວັດທະນະທຳ ຈ.ໃຫ້ ປຣອແກມໂຕນີ ໃຊ້ ສປຸ ນັ້ນ*
  - **APIs** for user programs to communicate desired activities to the OS.

# Memory (1)

- Is accessed all the time when a program is running.
  - Programs keep all data in memory, and accesses them through various instructions (lw, sw)
  - Programs' instructions are also stored in the memory

```
P1
main ()
{
    int *p= malloc (sizeof(int))
    *p = 0;
}
```

```
P2
main ()
{
    int *p= malloc (sizeof(int))
    *p = 0;
}
```

# Memory (2)

```
P1
main ()
{
    int *p= malloc (sizeof(int))
    *p = 0;
}
```

```
P2
main ()
{
    int *p= malloc (sizeof(int))
    *p = 0;
}
```

- Q: when P1 and P2 are running their program, it always appear that the value can be updated independently, why ?
  - Each running program has its own private memory (private address space), which can be mapped onto the physical memory
  - A memory reference within one program does not affect address space of others.
  - OS is virtualizing memory

# It's all about '*Concurrency*'

- OS has to <sup>จัดการ</sup> juggle many things at once, which leads to some interesting problems.
- Ultimate Goal: how to make all programs execute with correct conditions
  - What techniques are needed from the OS
  - What mechanisms should be provided by the hardware
  - How can we use them to solve the problems
- Multi-threaded programs also exhibit the same problems
  - Solutions are different from OS-level concurrency as threads share same address space.

# Persistence

- In memory, data can be easily lost, as DRAM is volatile.
  - Power out -> data lost
  - System crash -> data lost
- We need a disk drive for long-lived information.
- Thus, OS must manage the disk(s) in addition to other things. This is called a **file system**.
- The file system stores files in a reliable and efficient manner on the disk(s)
- No abstraction here, OS does not create a private, virtualized disk for each application, why ?

# Building an OS, we must consider ..

- Abstractions: make it convenient and easy to use through virtualization
- High performances: minimizing overheads (extra time, extra space)
- Isolation: protection schemes between applications (malicious or accidental bad behavior of one does not harm others)
- Reliability: OS runs non-stop. However, when it fails, all applications fail as well. Millions lines of code are dedicated to reliability.
- Energy-efficiency: green world, longer battery usage time
- Security: against malicious applications
- Mobility: for smaller devices

Operating system takes physical resources and virtualizes them. It also handles concurrency and persistence issues as well as others

Now... let's look at the history of OS

ได้ชื่อในยุคนั้น

# Early days of Mainframe

ทำงานได้ 1 program



- Early OS: just libraries of commonly used functions such as I/O manipulations
  - One program at a time, controlled by a human (human-scheduler).
- Libraries with Protection: code run on behalf of OS is special; it should be treated differently from others
  - Cannot allow any program to access anywhere on the disk (privacy).
  - Modes of execution is added: user and kernel
  - In contrast to a procedure call, a system call transfers control into the OS, raising the hardware privilege level. This allow full access to hardware of the system.
    - Initiate I/O requests
    - Make more memory available to a program

# Multiprogramming on Minicomputer

- To make better use of machine resources, people pay lots of attention to multiprogramming.
- Instead of running one job at a time, OS load a number of jobs into memory and swatch between them, improving CPU utilization.
- I/O is slow, doing this CPU does not have to wait.
- Issues arise:
  - Memory protection
  - Interrupts (must make sure OS is working correctly)
- Leads to the introduction of **UNIX** and with it the **C** language !  
OS ? δαυ Linux

# The Modern Era of PCs

The concept: “one machine per desktop instead of a shared minicomputer per workgroup”.

- People forgot to look at what're good in the minicomputer OS. So.....
- DOS: memory protection is absent
- Mac OS (1<sup>st</sup> gen): poor job scheduling

After some years of trying

- Windows NT incorporated features from minicomputer OS
- Mac OS X had UNIX at its core